Profile Picture Upload: Frontend Implementation Guide

This guide covers wiring profile-picture upload into the existing Mothercat frontend on both **My profile** and staff detail pages. It documents the current backend contract; it does not add a direct browser-to-Cloudinary upload flow.

Backend contract

Item	Contract
Upload	`POST /api/v1/staff/{staffId}/profile-picture`
`{staffId}`	The staff record UUID (`staff.id`), **not** the human-readable `staff.staff_id`
Request	`multipart/form-data` with exactly one file field named `file`
Accepted file	JPEG, PNG, or GIF; maximum 5 MiB; image dimensions are checked by the backend
Authentication	Existing `mothercat_session` cookie; same-origin mutation
Authorization	The account linked to the staff record may update its own picture. Other users need `staff.update`.
Success	HTTP 200 with `{ "staff": { ..., "profile_image_url": "https://res.cloudinary.com/..." } }`
Read	Existing `GET /api/v1/staff/{staffId}` and `GET /api/v1/staff` return `profile_image_url`

The backend sends image bytes to Cloudinary and saves the returned secure URL in the staff record. The frontend should upload to the backend and use the returned URL to display the image; do not call Cloudinary directly or expose Cloudinary credentials in the frontend.

The backend code and the database migration must be deployed before this feature can succeed in a deployed environment.

Existing frontend pieces to reuse

- `src/app/(workspace)/profile/page.tsx` renders the signed-in user's profile. It is a Server Component and currently has the signed-in session.
- `src/lib/auth/types.ts` represents `session.staffId` as the human-readable staff number and `session.userId` as the account UUID. The session does **not** currently contain the staff-record UUID or image URL.
- `src/components/workspace/staff-profile.tsx` loads a staff record with `getStaff(staffId)` and

already has the staff record UUID in `staff.id`.

- `src/lib/organization/api.ts` defines `StaffMember.profile_image_url`, parses the field, and has `uploadStaffCsv` as the existing `FormData` upload pattern.
- `src/lib/api/routes.ts` contains the API route builders.
- `src/lib/api/proxy.ts` forwards same-origin API calls and currently caps most request bodies at 16 KiB. It has a larger exception for CSV imports only.
- `src/components/workspace/dashboard-overview.tsx` already renders `profile_image_url` using `next/image` with `unoptimized` and an initials fallback. Reuse that approach or an equivalent existing avatar pattern.

Required implementation changes

1. Add the route builder and API helper

Add a profile-picture route builder in `src/lib/api/routes.ts`:

```
profilePicture: (staffId: string) => `/staff/${staffId}/profile-picture`,
```

In `src/lib/organization/api.ts`, follow the `uploadStaffCsv` pattern. The helper should append the selected `File` under `"file"`, POST to the route above, validate the `{ staff }` response with the existing `parseStaff`, and return the updated `StaffMember`.

```
const form = new FormData();
form.append("file", file);
const payload = await request(
  apiRoutes.staff.profilePicture(encodeURIComponent(staffId)),
  { method: "POST", body: form },
);
```

Use the existing `request` helper: it already includes credentials and handles API errors. **Do not set `Content-Type` yourself** for `FormData`; the browser must add the multipart boundary.

2. Increase the frontend proxy's upload body limit

The upload body currently gets rejected by the Next.js proxy's 16 KiB default before it reaches the backend. In `src/lib/api/proxy.ts`:

- Add a separate, named limit for profile-picture uploads: `5 * 1024 * 1024 + 64 * 1024` bytes. The extra 64 KiB is for the multipart envelope and matches the backend request cap.
- Select that limit only for the exact `staff/{UUID}/profile-picture` POST route. Keep the existing 16 KiB JSON limit and CSV import limit unchanged.
- Keep the existing same-origin check, session-cookie forwarding, content-type forwarding, and bounded streaming read. Do not remove the body limit.
- The proxy currently has a 15-second upstream timeout for ordinary requests, and the backend upload operation is also bounded. If real uploads hit timeouts, coordinate any timeout adjustment with the backend owner rather than removing timeouts.

The route handler at `src/app/api/v1/[...path]/route.ts` already forwards POST requests through `proxyApiRequest`; it should not need a new endpoint.

3. Wire the staff detail page

In `src/components/workspace/staff-profile.tsx`, use the already-loaded `staff.id` as the path ID. Show the upload/replace control only where the current user's permissions allow it (normally `permissions.canUpdate` for staff management). On success, replace local state with the `StaffMember` returned from the upload helper. Render `staff.profile_image_url` and retain the current initials fallback if it is null or the image fails to load.

4. Wire My profile without confusing staff IDs

The signed-in session on `src/app/(workspace)/profile/page.tsx` has `session.staffId` (a staff number), but the upload endpoint requires the staff record UUID. Do not put `session.staffId` directly into the endpoint path.

Resolve the current staff record first, for example by calling the existing `listStaff` helper with a search for `session.staffId`, then selecting only the record whose `staff_id` exactly equals `session.staffId`. Use that record's `id` for upload and its `profile_image_url` for display. If the record is missing or the lookup fails, show an actionable error and do not guess an ID or try uploading to another record. The backend still enforces that the signed-in user owns the record.

The profile page is server-rendered today. Keep session access server-side and add a focused Client Component for file selection, upload state, preview, and feedback if interaction is needed; pass only the necessary profile values to it.

5. Validate and present upload state

- Use a file input restricted with `accept="image/jpeg,image/png,image/gif"`.
- Check the selected file is non-empty, one of those MIME types, and at most 5 MiB before uploading. The server remains authoritative; the input's `accept` attribute is not validation.
- If showing a local preview, use an object URL and revoke it when replaced or when the component unmounts.
- Disable duplicate submissions; communicate upload progress/state and success/failure with accessible status and alert text.
- On success, display the persisted URL returned by the server, not just the temporary preview. Refreshing/reloading should show the same image from the normal staff read response.
- Retain initials when no image exists or an image cannot load. Add suitable alt text (or empty alt for a decorative avatar when the name is already adjacent).
- Map backend errors through `formatOrganizationError` and give specific, understandable feedback for invalid image, unsupported type, size too large, missing staff record, permission denied, and upload/service failure. Treat proxy `413 request_too_large` as the upload-size limit being exceeded.

Error behavior to account for

Response/code	Frontend behavior
`400 invalid_image` / `unsupported_image_type`	Ask the user to select a valid JPEG, PNG, or GIF image

Response/code	Frontend behavior
`413 image_too_large` / proxy `request_too_large`	Explain the 5 MiB file limit
`401 unauthorized`	Use the existing expired-session/sign-in handling
`403 permission_denied`	Explain that the user cannot edit this profile
`404 staff_not_found`	Refresh the profile or report that the staff record could not be found
`502 image_upload_failed`, `503 service_unavailable`, or network/timeout error	Keep the prior image, show a retryable error, and retain any support request ID

Do not silently treat an upload as successful if the response is malformed or fails.

Verification checklist

- API-client tests assert the URL, POST method, `FormData` field name, credentials, and that no explicit multipart `Content-Type` is set.
- Proxy tests assert a valid profile-picture body just above 16 KiB is forwarded, a body over 5 MiB + 64 KiB is rejected with 413, and the CSV and JSON limits remain unchanged.
- UI tests cover self-profile UUID resolution, exact staff-number matching, missing lookup result, preview/fallback, successful replacement, duplicate-submit prevention, and each user-visible failure state.
- Manually upload a valid image, confirm it appears on the staff profile and My profile, reload and confirm it persists, then verify invalid type and oversized files are rejected.

Relevant files

- `src/app/(workspace)/profile/page.tsx`
- `src/components/workspace/staff-profile.tsx`
- `src/lib/organization/api.ts`
- `src/lib/api/routes.ts`
- `src/lib/api/proxy.ts`
- `src/app/api/v1/[...path]/route.ts`
- `src/components/workspace/dashboard-overview.tsx`